

LAB 10

Task # 01

```
class Main{
    private String outerMessage = "Hello from the Outer Class!";
    class InnerClass {

        // Instance variable of the inner class
        private String innerMessage = "Hello from the Inner Class!";

        // Method of InnerClass
        public void displayMessages() {
            // Accessing the outer class's member from the inner class
            System.out.println(outerMessage); // Access outer class's instance variable
            System.out.println(innerMessage); // Access inner class's instance variable
        }
    }

    // Method to create an instance of the InnerClass from within OuterClass
    public void createInnerClassInstance() {
        // Create an instance of InnerClass from within OuterClass
        InnerClass innerClass = new InnerClass();
        // Call the method of InnerClass
        innerClass.displayMessages();
    }

    public static void main(String[] args) {
        // Create an instance of OuterClass
        Main outerClass = new Main();

        // Create an instance of InnerClass and access its members through OuterClass
        outerClass.createInnerClassInstance();
    }
}
```

Task # 02

```
// OuterClass with a Static Nested Class
public class Main {
    // Static variable of OuterClass
    static String outerMessage = "Hello from the Outer Class (Static)!";

    // Static Nested Class
    static class StaticNestedClass {
```

LAB 10

```
// Method in StaticNestedClass
public void displayMessage() {
    // Accessing static member of OuterClass
    System.out.println(outerMessage); // Can access static member of OuterClass
}
}

public static void main(String[] args) {
    // You don't need to create an instance of OuterClass to create an instance of StaticNestedClass

    // Create an instance of StaticNestedClass directly using OuterClass.StaticNestedClass
    Main.StaticNestedClass nestedClass = new Main.StaticNestedClass();

    // Call the method of StaticNestedClass
    nestedClass.displayMessage(); // Output: Hello from the Outer Class (Static)!
}
}
```

Task # 03

```
// OuterClass with a member inner class
public class Main {
    // Instance variable of OuterClass
    private String outerMessage = "Hello from the Outer Class!";

    // Instance method of OuterClass
    public void outerMethod() {
        System.out.println("This is the outer method.");
    }

    // Member Inner Class
    class MemberInnerClass {
        // Method in the inner class to access members of the outer class
        public void displayOuterMessage() {
            // Accessing non-static member of the OuterClass (outerMessage)
            System.out.println(outerMessage); // Access the instance variable of the outer class

            // Accessing method of the OuterClass
            outerMethod(); // Calling the outer method
        }
    }

    // Method in OuterClass to create an instance of MemberInnerClass
    public void createInnerClassInstance() {
        // Create an instance of MemberInnerClass within the method of OuterClass
    }
}
```

LAB 10

```
MemberInnerClass innerClass = new MemberInnerClass();

// Call the displayOuterMessage method of MemberInnerClass
innerClass.displayOuterMessage();
}

public static void main(String[] args) {
    // Create an instance of OuterClass
    Main outerClass = new Main();

    // Call the createInnerClassInstance method to create and use the MemberInnerClass
    outerClass.createInnerClassInstance();
}
}
```

Task # 04

```
public class Main {
    // Method in OuterClass that contains a local inner class
    public void outerMethod() {
        // Local variable inside the method
        final String message = "Hello from the local inner class!";

        // Local inner class defined within the outerMethod
        class LocalInnerClass {
            // Method to display the message from the local variable
            public void displayMessage() {
                // Accessing the local variable from the outer method
                System.out.println(message);
            }
        }

        // Create an instance of the local inner class inside the method
        LocalInnerClass innerClass = new LocalInnerClass();
        // Call the displayMessage method of LocalInnerClass
        innerClass.displayMessage();
    }

    public static void main(String[] args) {
        // Create an instance of OuterClass
        Main outerClass = new Main();

        // Call the outerMethod, which contains the local inner class
        outerClass.outerMethod();
    }
}
```

LAB 10

```
}  
}
```

Task # 05

```
class Animal {  
    public void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // Anonymous inner class that extends Animal class  
        Animal myAnimal = new Animal() {  
            // Overriding the makeSound method of Animal class  
            @Override  
            public void makeSound() {  
                System.out.println("Dog barks");  
            }  
        };  
  
        // Calling the overridden method of the anonymous inner class  
        myAnimal.makeSound(); // Output: Dog barks  
    }  
}
```

Key Points

- Nested classes provide a way to organize code and create relationships between classes.
- Inner classes have access to the members of the outer class, including private members.
- Static nested classes can be accessed without creating an instance of the outer class.
- Member inner classes are associated with an instance of the outer class.
- Local inner classes are defined within a method and have limited scope.
- Anonymous inner classes are used to create unnamed classes on the fly.